Tyr: a new Rust GPU driver

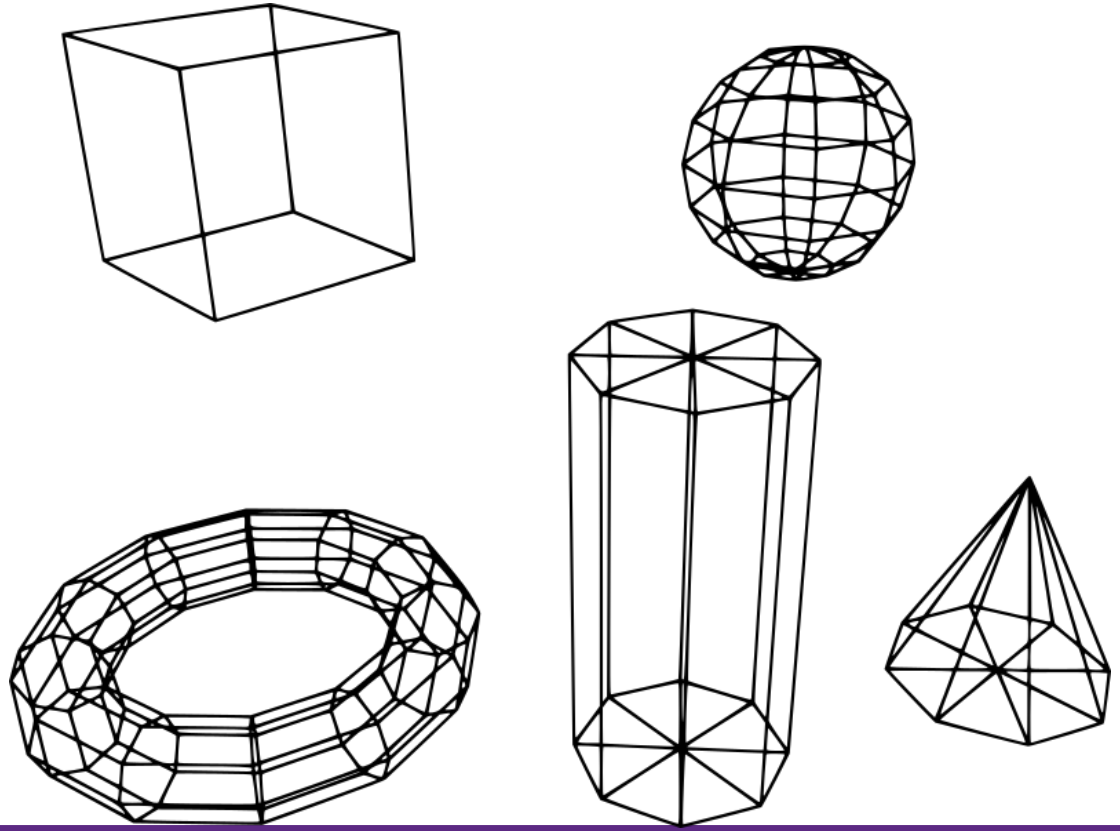
Alice Ryhl (Google)

Daniel Almeida (Collabora)

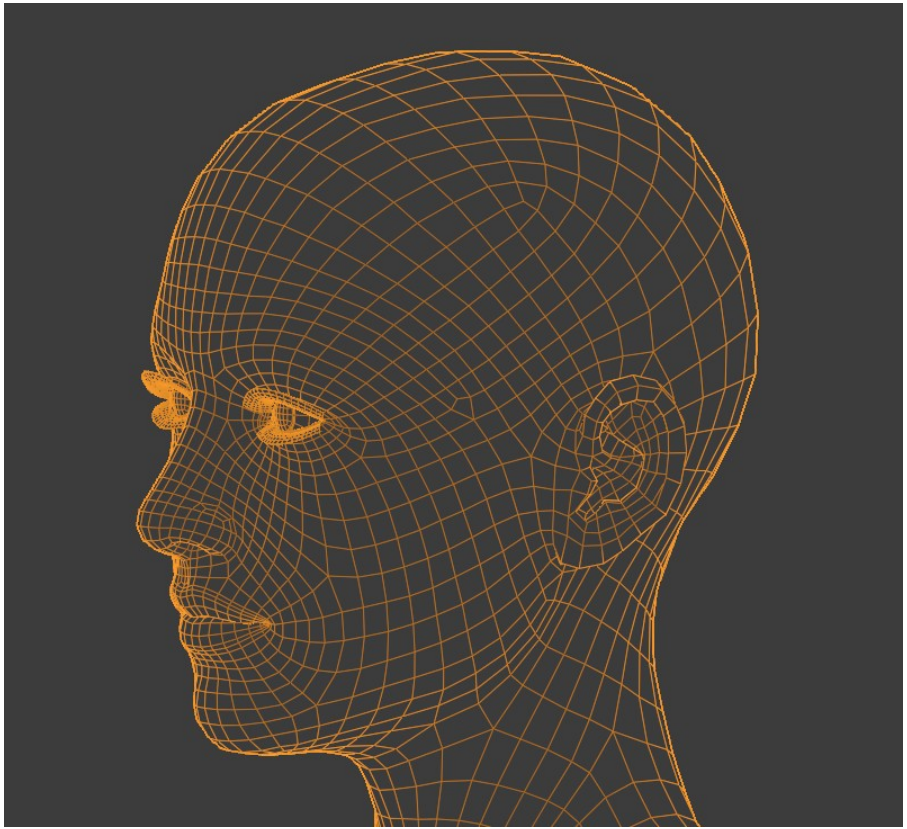
In this talk..

- Briefly discuss how GPU drivers work
- Discuss the KMD uAPI
- CSF
- Abstractions
- Prototype
- Future plans

Simple meshes

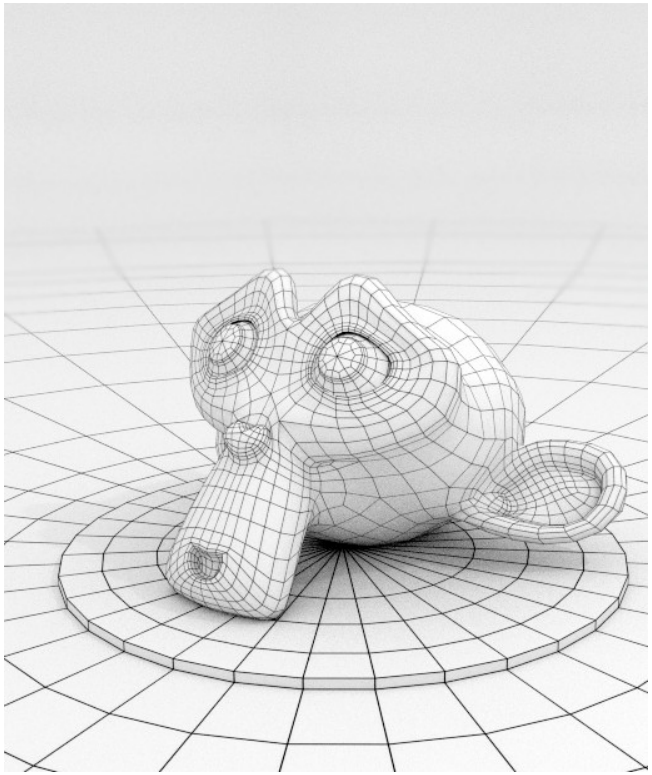


More elaborate meshes



https://blender.stackexchange.com/questions/213611/mesh-looks-ok-in-wireframe-but-shading-is-distorted

More elaborate meshes



https://blenderartists.org/t/suzanne-on-wire/678653

Textures



https://www.poliigon.com/texture/reclaimed-dutch-bond-brick-wall-texture/8320

Shaders

- Full blown programs
- At the very minimum, places the object on scene
- May apply any number of effects, like e.g.: rotation
- Executed by the shader cores in the GPU

Shaders

```
/* vkcube.vert */
#version 420 core

layout(std140, set = 0, binding = 0) uniform block {
    uniform mat4 modelviewMatrix;
    uniform mat4 modelviewprojectionMatrix;
    uniform mat3 normalMatrix;
};

layout(location = 0) in vec4 in_position;
layout(location = 1) in vec4 in_color;
layout(location = 2) in vec3 in_normal;

vec4 lightSource = vec4(2.0, 2.0, 20.0, 0.0);

layout(location = 0) out vec4 vVaryingColor;

void main()
{
    gl_Position = modelviewprojectionMatrix * in_position;
    vec3 vEyeNormal = normalMatrix * in_normal;
    vec4 vPosition4 = modelviewMatrix * in_position;
    vec3 vPosition3 = vPosition4.xyz / vPosition4.w;
    vec3 vLightDir = normalize(lightSource.xyz - vPosition3);
    float diff = max(0.0, dot(vEyeNormal, vLightDir));
    vVaryingColor = vec4(diff * in_color.rgb, 1.0);
}
```

Shaders

```
/* vkcube.frag */
#version 420 core

layout(location = 0) in vec4 vVaryingColor;
layout(location = 0) out vec4 f_color;

void main()
{
    f_color = vVaryingColor;
}
```



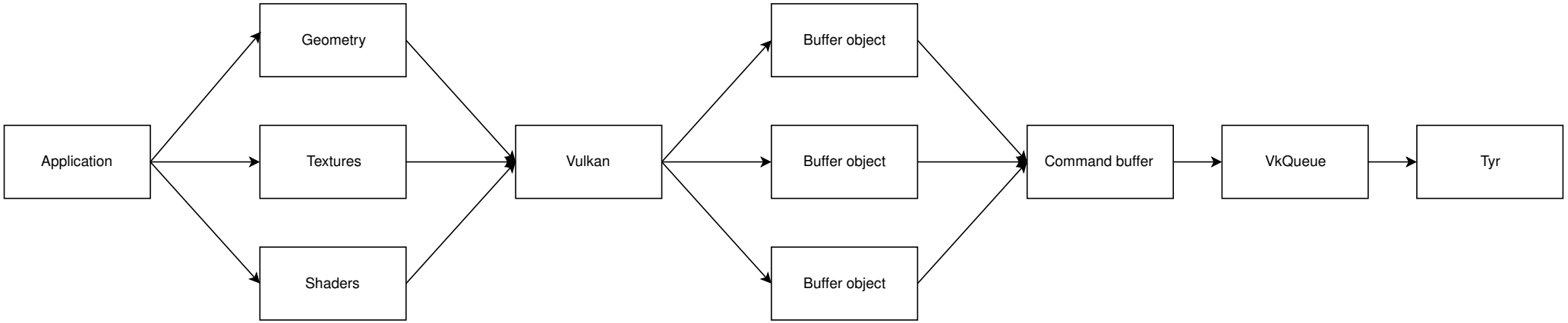
COLLABORA

Everything is a buffer

Buffer objects

- Vertex data → Vertex BO
- Texture data → Texture BO
- Pipeline configuration → Command Stream BO
- Shader machine code → Shader BO

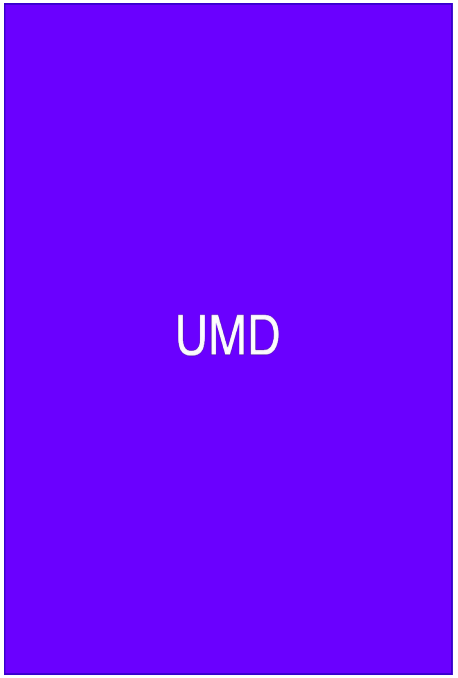
Overview


Application
Geometry
Textures
Shaders
Vulkan
Buffer object
Buffer object
Buffer object
Command buffer
VkQueue
Tyr

Code repartition



Code repartition



- Responsible for translating API (Vulkan, OpenGL, OpenCL, etc) calls into HW-specific requests
- Shader compilation
- Pipeline configuration

Code repartition

KMD

- Here to assist the UMD:
- Memory management and isolation,
- Job submission and dependency tracking
- Synchronization
- Power management

Code repartition



- Debugging facilities of all sorts
- Error recovery (very important!)
- Bridge between the UMD and the hardware

Memory isolation

- Each GPU context gets its own address space (called VM)
- VM isolation is enforced by the IOMMU
- UMD is in charge of mapping/unmapping memory in the VM
- With power, comes responsibility: mismanagement by the UMD will cause faults which will make Tyr kill the context

Synchronization

- GPUs have a lot of work to do and CPU ↔️ GPU round-trips are relatively slow
- Complex frames require multiple passes with multiple internal/external dependencies
- Memory resources can be shared with other device/processes → need to wait for other operations to complete



COLLABORA

The KMD must inform the UMD when jobs complete or the UMD may wait forever!

Job submission and dep. tracking

- The UMD will effectively submit a dependency graph
- It’s up to the KMD to ensure that this dependency graph is actually followed
- In other words, a job can only be scheduled once its dependencies have completed.
- It’s up to the KMD to enforce this.



COLLABORA

Asynchronously checking for job completion is key to performance

Error recovery

- If the GPU crashes, this will take down all graphical apps
- The KMD has to take steps to ensure that faults are handled
- If faults are fatal, the KMD might try to reset the GPU and restart the hardware in a clean way
- If this is not implemented, the machine will need a reboot

Power management

- Mali GPUs are usually used in an embedded/mobile context where power is at a premium
- It’s up to the KMD to enforce different power states depending on the different workload levels
- A GPU can work without this, but it will operate inefficiently. It can be perhaps unsuitable for day to day use.

Debug facilities

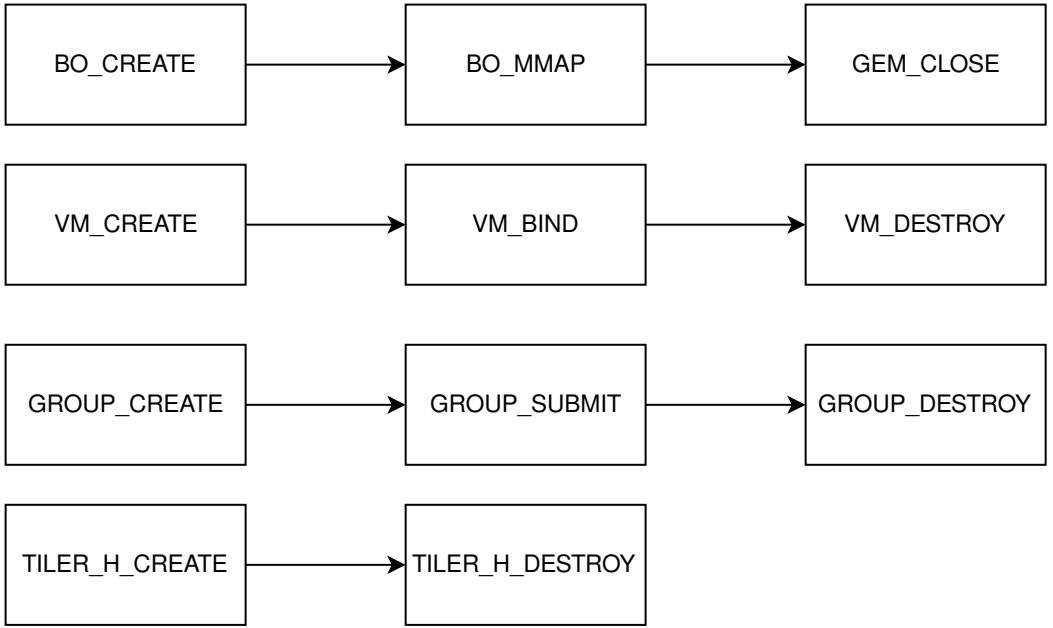
- If something goes wrong, how do we debug it?
- The UMD may make mistakes from time to time
- The KMD can also have bugs!
- It’s up to the KMD to export (usually vendor-specific) information to debug issues



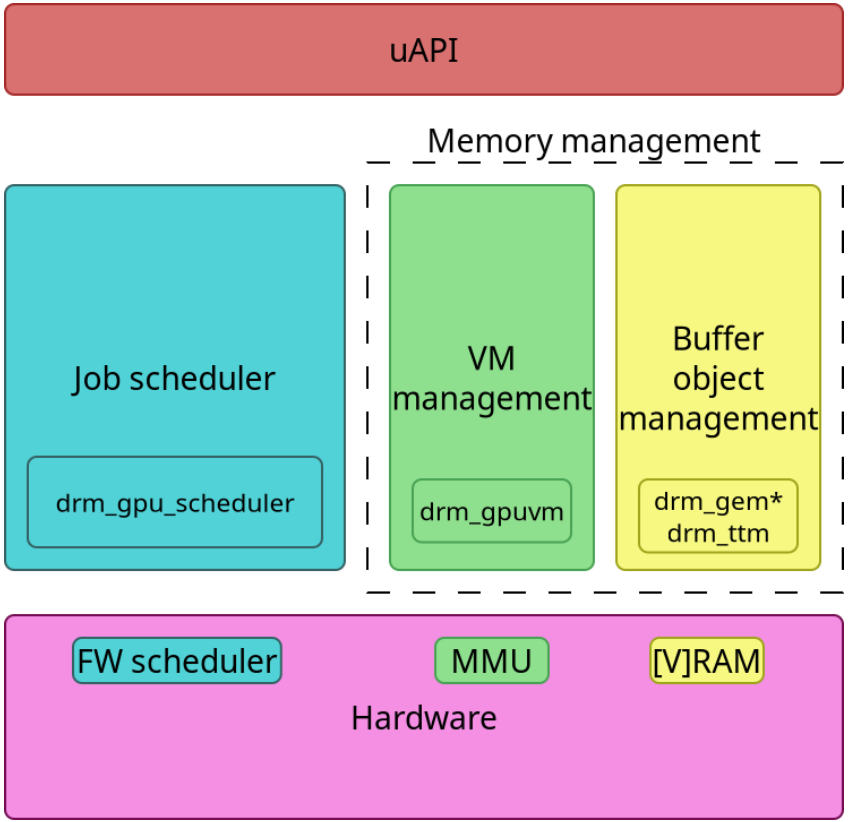
COLLABORA

The UMD is very complex, but the KMD offers a much simpler API

Kernel driver userspace API



A GPU KMD in a nutshell

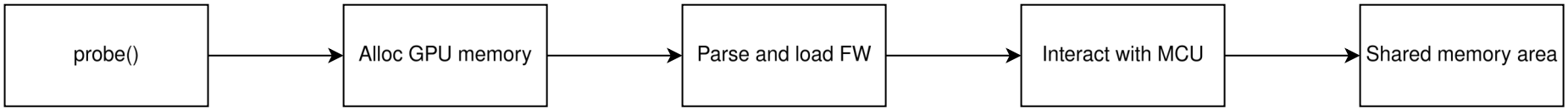




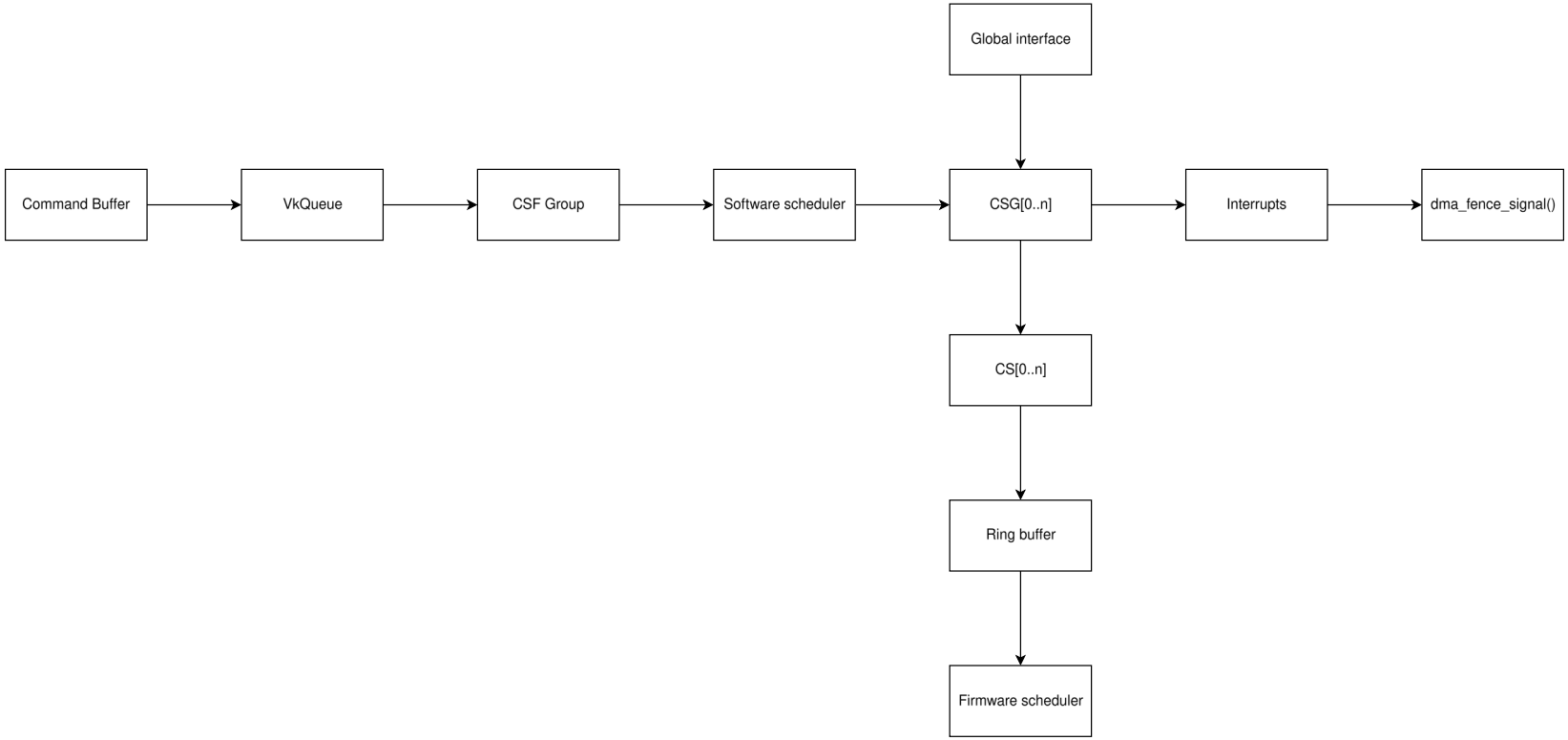
COLLABORA

CSF: hardware-assisted job scheduling

Booting the CSF MCU



Job submission and execution on CSF





COLLABORA

The infrastructure, i.e.: Rust abstractions

Basic infrastructure in upstream

- drm_device, drm_file, etc → Interface with DRM ✅
- Clk, Regulator → Power up the HW ✅
- IRQs → Job completion and more ✅
- Workqueues → run deferred work ✅

Basic infrastructure in downstream

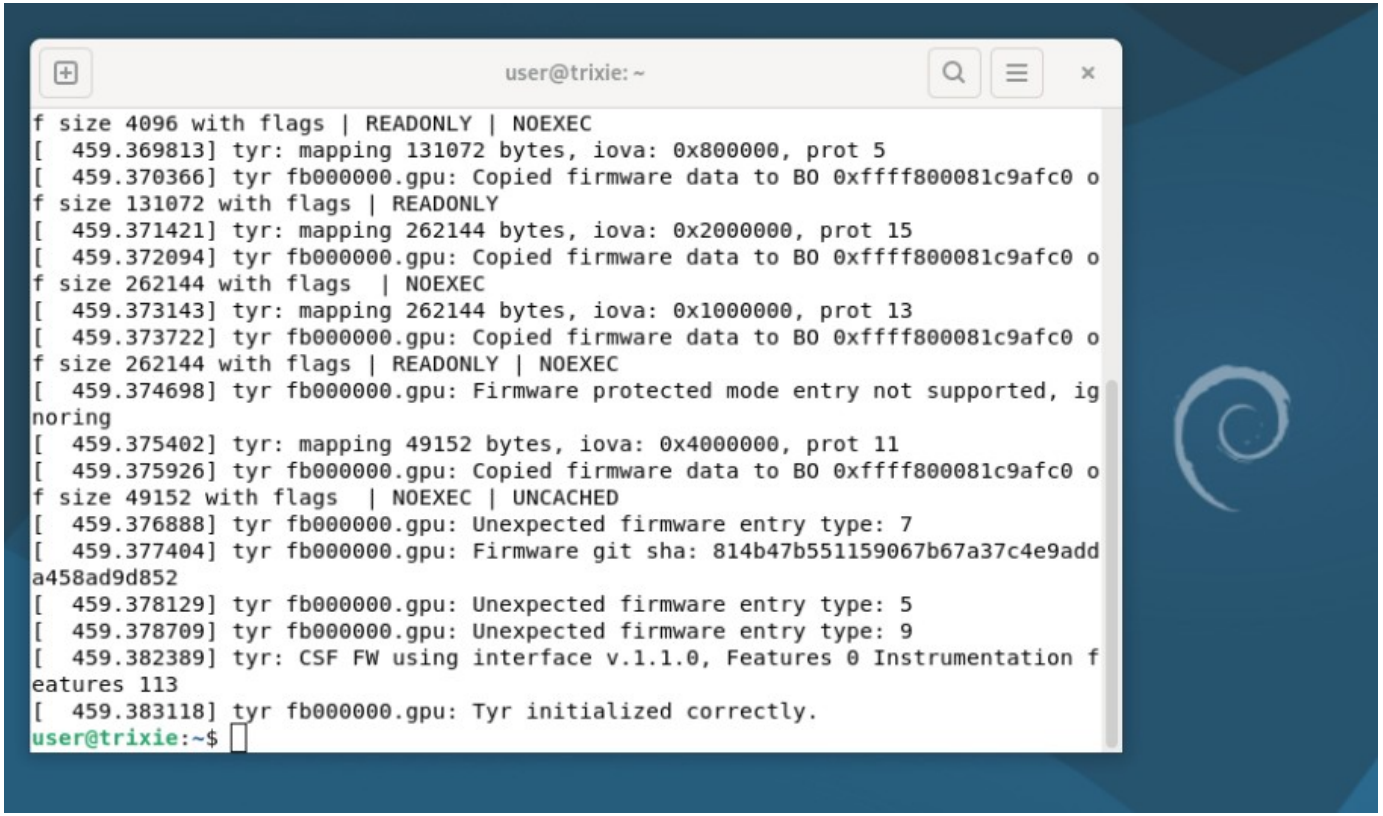
- drm_gem_shmem, drm_gpuvm → Allocate GPU memory
- io_pgtable → Communicate with the IOMMU
- dma_fence, drm_syncobj → Job synchronization
- drm_gpu_scheduler* → Dependency resolution



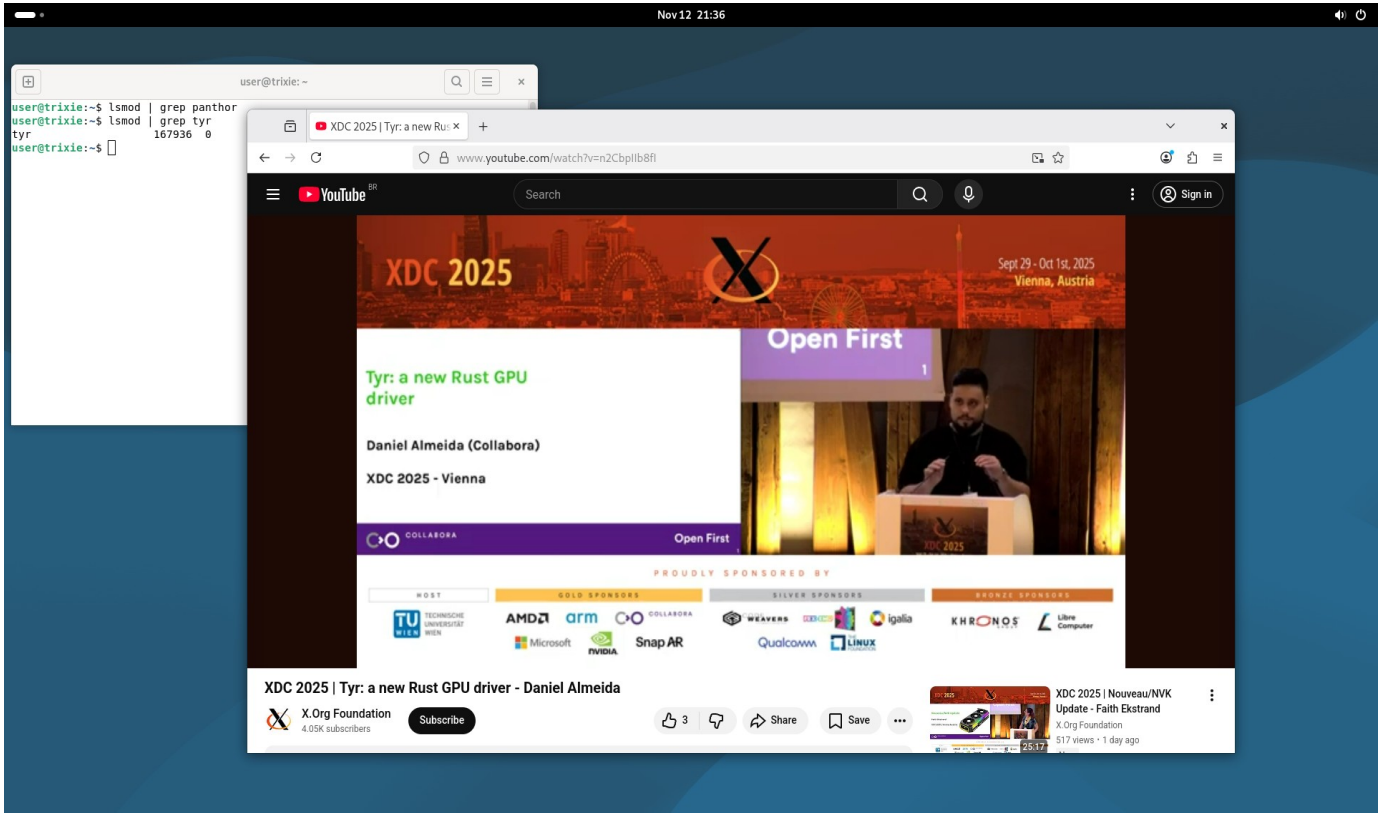
COLLABORA

Let’s talk about the downstream prototype

Prototype


user@trixie: ~
f size 4096 with flags | READONLY | NOEXEC
[459.369813] tyr: mapping 131072 bytes, iova: 0x800000, prot 5
[459.370366] tyr fb000000.gpu: Copied firmware data to BO 0xffff800081c9afc0 o
f size 131072 with flags | READONLY
[459.371421] tyr: mapping 262144 bytes, iova: 0x2000000, prot 15
[459.372094] tyr fb000000.gpu: Copied firmware data to BO 0xffff800081c9afc0 o
f size 262144 with flags | NOEXEC
[459.373143] tyr: mapping 262144 bytes, iova: 0x1000000, prot 13
[459.373722] tyr fb000000.gpu: Copied firmware data to BO 0xffff800081c9afc0 o
f size 262144 with flags | READONLY | NOEXEC
[459.374698] tyr fb000000.gpu: Firmware protected mode entry not supported, ig
noring
[459.375402] tyr: mapping 49152 bytes, iova: 0x4000000, prot 11
[459.375926] tyr fb000000.gpu: Copied firmware data to BO 0xffff800081c9afc0 o
f size 49152 with flags | NOEXEC | UNCACHED
[459.376888] tyr fb000000.gpu: Unexpected firmware entry type: 7
[459.377404] tyr fb000000.gpu: Firmware git sha: 814b47b551159067b67a37c4e9add
a458ad9d852
[459.378129] tyr fb000000.gpu: Unexpected firmware entry type: 5
[459.378709] tyr fb000000.gpu: Unexpected firmware entry type: 9
[459.382389] tyr: CSF FW using interface v.1.1.0, Features 0 Instrumentation f
eatures 113
[459.383118] tyr fb000000.gpu: Tyr initialized correctly.
user@trixie:~$

Prototype


Nov 12 21:36
user@trixie: ~
user@trixie:~$ lsmod | grep panthor
user@trixie:~$ lsmod | grep tyr
tyr 167936 0
user@trixie:~$
XDC 2025 | Tyr: a new Ru
www.youtube.com/watch?v=n2CbpIIb8fI
YouTube
Search
Sign in
XDC 2025
Sept 29 - Oct 1st, 2025
Vienna, Austria
Tyr: a new Rust GPU driver
Daniel Almeida (Collabora)
XDC 2025 - Vienna
Open First
PROUDLY SPONSORED BY
XDC 2025 | Tyr: a new Rust GPU driver - Daniel Almeida
X.Org Foundation
4.05K subscribers
Subscribe
Share
Save
XDC 2025 | Nouveau/NVK Update - Faith Ekstrand
X.Org Foundation
517 views • 1 day ago

Prototype


00:41.158
+00:02.475
+00:02.475
+00:02.475
00:43.633
TUX
4
New fastest lap
00:41.158 by Emule

Prototype

- GNOME, Weston, 3D apps are working downstream
- A lot can be upstreamed once some Rust bindings land
- Some parts will need more iterations before being ready
- Can be used as a testbed for shared GPU infrastructure
- This will help other drivers like Nova and AGX



COLLABORA

There’s a prototype, but what about the future?

Future plans

- The prototype works, can we make it better?
- How can we rethink the design to eliminate shortcuts?
- How can we leverage Rust's safety guarantees even more?
- How is job submission going to look like?
- What can we currently upstream?

Future plans so far

- Rethink design for upstreaming
- Serve as a testbed for shared infrastructure
- Help with the JobQueue development as needed
- Benchmark as appropriate


Race karts on Tyr, the new Rust DRM driver for Arm Mali
Running on Radxa Rock 5B
Powered by Rockchip RK3588
Developed in a joint effort between
COLLABORA
arm
Google
東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025
Steering
Reverse
Accelerate
Use item



COLLABORA

Questions? Discussions?



COLLABORA

Come race with us at our booth on 4th floor!

Race karts on Tyr, the new Rust DRM driver for Arm Mali

Running on Radxa Rock 5B
Powered by Rockchip RK3588

Developed in a joint effort between

COLLABORA arm Google



東京 2025
LINUX PLUMBERS CONFERENCE
TOKYO, JAPAN / DEC. 11-13, 2025



Thank you!